

**LAPORAN PRAKTIKUM PEMROGRAMAN WEB MODUL 12:
MEMBUAT APLIKASI REACT BLOG POST
MENGUNAKAN API**



**Universitas
Telkom**

Disusun Oleh:
Nama: Andhika Yuridho
NIM: 707012500059
Kelas: D4SIKC-49-04

**UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026**

Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep integrasi frontend React dengan backend REST API berbasis CodeIgniter 3.
2. Membuat aplikasi React Blog Post yang mengambil data artikel dari endpoint API.
3. Menggunakan Axios untuk mengirim request HTTP dan mengelola response JSON dari server.
4. Menerapkan autentikasi sederhana menggunakan endpoint login, token JWT, dan penyimpanan token pada localStorage.
5. Membangun halaman login, daftar post, dan detail post menggunakan React Router.
6. Mengatur proxy Vite agar frontend React dapat berkomunikasi dengan backend CodeIgniter 3 saat development.
7. Menampilkan data post, gambar, author, dan tanggal secara dinamis berdasarkan data dari database.
8. Menguji hasil integrasi melalui browser dan endpoint API JSON.

Alat dan Bahan

Untuk menjalankan modul ini, diperlukan komputer atau laptop dengan sistem operasi Windows serta browser modern seperti Google Chrome atau Microsoft Edge.

Perangkat lunak yang digunakan meliputi:

1. Node.js dan npm sebagai runtime serta package manager untuk menjalankan project React.
2. React dan React Router DOM untuk membangun halaman login, daftar post, dan detail post.
3. Axios untuk melakukan komunikasi HTTP dari frontend ke REST API.
4. Vite sebagai development server dan build tool aplikasi React.
5. Laragon sebagai web server lokal untuk menjalankan project CodeIgniter 3 dan database MySQL.
6. CodeIgniter 3 HMVC sebagai backend REST API.
7. Visual Studio Code atau code editor lain untuk mengelola file program.

Bahan yang dibutuhkan:

1. Project backend-ci3(pekan9) sebagai backend REST API.
2. Project frontend-reactjs sebagai aplikasi React Blog Post.
3. Database ci3 project beserta tabel users dan posts.
4. File konfigurasi API, route, controller, model, dan komponen React.

Langkah-Langkah Praktikum

Menyiapkan folder praktikum Modul 12 yang berisi backend-ci3 dan frontend-reactjs.

Mengimport file ci3_project.sql ke MySQL agar tabel users dan posts tersedia.

Mengatur konfigurasi database CodeIgniter 3 agar terhubung ke database ci3_project.

Membuat endpoint autentikasi register, login, logout, dan me pada module Api.

Membuat endpoint post untuk mengambil daftar post, detail post, membuat, mengubah, dan menghapus post.

Membuat library Jwt untuk menghasilkan dan memverifikasi token Bearer.

Membuat service Axios pada React untuk memusatkan base URL API dan header Authorization.

Membuat AuthContext untuk menyimpan user, token, proses login, dan proses logout.

Membuat route React untuk halaman login, daftar post, dan detail post dengan ProtectedRoute.
 Membuat tampilan PostList dan PostDetail yang membaca data dari API lalu menampilkannya secara dinamis.
 Menjalankan frontend menggunakan npm run dev dan backend melalui Laragon.
 Menguji endpoint API dan tampilan aplikasi melalui browser.

Hasil dan Dokumentasi Program

Berikut kode inti aplikasi React Blog Post yang terhubung dengan REST API CodeIgniter 3:

File: frontend-reactjs/package.json

```
{
  "name": "modul-13-react",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.13.4",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.13.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "autoprefixer": "^10.4.24",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.19",
    "vite": "^7.2.4"
  }
}
```

File: frontend-reactjs/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// https://vite.dev/config/
export default defineConfig({
  base: "./",
  plugins: [react()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost",
        changeOrigin: true,
        rewrite: (path) =>
          path.replace(/^\/api/, "WebPro4904/Pekan-12/backend-ci3/api"),
      },
    },
  },
});
```

File: frontend-reactjs/src/services/api.js

```
import axios from "axios";
```

```

const API_URL = import.meta.env.VITE_API_BASE_URL || "/api";

const api = axios.create({
  baseURL: API_URL,
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
});

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  },
);

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      const requestUrl = error.config?.url || "";

      if (!requestUrl.includes("/login") && !requestUrl.includes("/register")) {
        localStorage.removeItem("token");
        localStorage.removeItem("user");
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  },
);

export const authAPI = {
  login: (email, password) => api.post("/auth/login", { email, password }),
  register: (name, email, password, password_confirmation) =>
    api.post("/auth/register", {
      name,
      email,
      password,
      password_confirmation,
    }),
  logout: () => api.post("/auth/logout"),
  me: () => api.get("/auth/me"),
};

export const postsAPI = {
  getAll: (page = 1, perPage = 10) =>
    api.get(`/post?page=${page}&per_page=${perPage}`),
  getById: (id) => api.get(`/post/${id}`),
};

export default api;

```

File: frontend-reactjs/src/App.jsx

```

import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import ProtectedRoute from './components/ProtectedRoute';
import Login from './pages/Login';
import PostList from './pages/PostList';
import PostDetail from './pages/PostDetail';
import './index.css';

function App() {
  return (
    <AuthProvider>
      <Router>
        <Routes>
          <Route path="/login" element={<Login />} />

```

```

        <Route
          path="/posts"
          element={
            <ProtectedRoute>
              <PostList />
            </ProtectedRoute>
          }
        />
        <Route
          path="/posts/:id"
          element={
            <ProtectedRoute>
              <PostDetail />
            </ProtectedRoute>
          }
        />

        <Route path="/" element={<Navigate to="/posts" replace />} />
        <Route path="*" element={<Navigate to="/posts" replace />} />
      </Routes>
    </Router>
  </AuthProvider>
);
}

export default App;

```

File: frontend-reactjs/src/context/AuthContext.jsx

```

import { createContext, useContext, useState, useEffect } from 'react';
import { authAPI } from '../services/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const checkAuth = async () => {
      const savedToken = localStorage.getItem('token');
      const savedUser = localStorage.getItem('user');

      if (savedToken && savedUser) {
        setToken(savedToken);
        setUser(JSON.parse(savedUser));
      }
      setLoading(false);
    };

    checkAuth();
  }, []);

  const login = async (email, password) => {
    try {
      const response = await authAPI.login(email, password);
      const { access_token, data: userData } = response.data;

      localStorage.setItem('token', access_token);
      localStorage.setItem('user', JSON.stringify(userData));

      setToken(access_token);
      setUser(userData);

      return { success: true };
    } catch (error) {
      const message = error.response?.data?.message || 'Login gagal';
      return { success: false, message };
    }
  };

  const logout = async () => {
    try {

```

```

        await authAPI.logout();
    } catch (error) {
        console.error('Logout error:', error);
    } finally {
        localStorage.removeItem('token');
        localStorage.removeItem('user');
        setToken(null);
        setUser(null);
    }
};

const value = {
    user,
    token,
    loading,
    isAuthenticated: !!token,
    login,
    logout,
};

return (
    <AuthContext.Provider value={value}>
        {children}
    </AuthContext.Provider>
);
};

export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error('useAuth harus digunakan di dalam AuthProvider');
    }
    return context;
};

```

File: frontend-reactjs/src/pages/Login.jsx

```

import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const Login = () => {
    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [error, setError] = useState('');
    const [loading, setLoading] = useState(false);

    const { login } = useAuth();
    const navigate = useNavigate();

    const handleSubmit = async (e) => {
        e.preventDefault();
        setError('');
        setLoading(true);

        try {
            const result = await login(email, password);

            if (result.success) {
                navigate('/posts');
            } else {
                setError(result.message);
            }
        } catch (err) {
            console.error('Login error:', err);
            setError(err.message || 'Terjadi kesalahan saat login. Pastikan server CodeIgniter sudah berjalan.');
```

sudah berjalan.');

```

        } finally {
            setLoading(false);
        }
    }

    return (
        <div className="min-h-screen bg-gray-100 flex flex-col">
            <div className="flex-1 flex flex-col items-center justify-center px-6 py-8">
                <a href="#" className="flex items-center mb-8 text-2xl font-semibold text-
```

```

gray-900">
    <svg className="w-10 h-10 mr-3 text-blue-600" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
        <path d="M5 3a2 2 0 00-2 2v2a2 2 0 002 2h2a2 2 0 002-2v5a2 2 0 00-2-
2H5zM5 11a2 2 0 00-2 2v2a2 2 0 002 2h2a2 2 0 002-2v-2a2 2 0 00-2-2H5zM11 5a2 2 0 012-2h2a2 2 0
012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V5zM14 11a1 1 0 011 1v1h1a1 1 0 011 1h-1v1a1 1 0 01-1-1z"></path>
    </svg>
    Blog Posts
</a>

<div className="w-full max-w-md bg-white rounded-xl shadow-lg border border-
gray-200 p-8">
    <h1 className="text-2xl font-bold text-gray-900 mb-6">
        Sign in to your account
    </h1>

    {error && (
        <div className="mb-6 p-4 text-sm text-red-700 bg-red-100 rounded-lg
border border-red-200" role="alert">
            {error}
        </div>
    )}

    <form className="space-y-6" onSubmit={handleSubmit}>
        <div>
            <label htmlFor="email" className="block mb-2 text-sm font-medium
text-gray-900">
                Your email
            </label>
            // ... bagian kode lain mengikuti pola yang sama ...
            <div>
                <label htmlFor="password" className="block mb-2 text-sm font-
medium text-gray-900">
                    Password
                </label>
                <input
                    type="password"
                    name="password"
                    id="password"
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    placeholder="....."
                    className="w-full px-4 py-3 bg-gray-50 border border-gray-300
rounded-lg text-gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500 focus:border-
blue-500 outline-none transition-all"
                    required
                />
            </div>

            <div className="flex items-center justify-between">
                <div className="flex items-center">
                    <input
                        id="remember"
                        type="checkbox"
                        className="w-4 h-4 border border-gray-300 rounded bg-gray-
50 focus:ring-2 focus:ring-blue-500"
                    />
                    <label htmlFor="remember" className="ml-2 text-sm text-gray-
600">
                        Remember me
                    </label>
                </div>
                <a href="#" className="text-sm font-medium text-blue-600
hover:text-blue-500">
                    Forgot password?
                </a>
            </div>

            <button
                type="submit"
                disabled={loading}
                className="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 text-
white font-medium rounded-lg transition-colors focus:ring-4 focus:ring-blue-300
disabled:opacity-50 disabled:cursor-not-allowed"
            >
                {loading ? 'Signing in...' : 'Sign in'}
        </form>
    </div>

```

```

        </button>

        <p className="text-sm text-gray-600 text-center">
            Don't have an account yet?{' '}
            <a href="#" className="font-medium text-blue-600 hover:text-blue-
500">
                Sign up
            </a>
        </p>
    </form>
</div>
</div>

    <footer className="bg-white border-t border-gray-200 py-4">
        <p className="text-center text-sm text-gray-500">
            © 2024 Blog Posts. All rights reserved.
        </p>
    </footer>
</div>
    );
};

export default Login;

```

File: frontend-reactjs/src/pages/PostList.jsx

```

import { useState, useEffect } from 'react';
import { postsAPI } from '../services/api';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import PostCard from '../components/PostCard';

const PostList = () => {
    const [posts, setPosts] = useState([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState('');

    useEffect(() => {
        fetchPosts();
    }, []);

    const fetchPosts = async () => {
        try {
            setLoading(true);
            const response = await postsAPI.getAll();
            setPosts(response.data.data);
            setError('');
        } catch (err) {
            setError('Gagal memuat posts. Silakan coba lagi.');
```

console.error('Error fetching posts:', err);
 } finally {
 setLoading(false);
 }
 };

 return (
 <div className="min-h-screen bg-gray-100 flex flex-col">
 <Navbar />

 <main className="flex-1">
 <div className="py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
 <div className="mx-auto max-w-screen-sm text-center lg:mb-16 mb-8">
 <h2 className="mb-4 text-3xl lg:text-4xl tracking-tight font-extrabold
text-gray-900">
 Our Blog
 </h2>
 <p className="font-light text-gray-500 sm:text-xl">
 Explore our collection of articles about React, web development,
and more.
 </p>
 </div>

 {loading ? (
 <div className="flex justify-center items-center py-20">
 <div className="animate-spin rounded-full h-12 w-12 border-b-2
border-blue-600"></div>


```

        </div>
      ) : error ? (
        <div className="text-center py-20">
          <p className="text-red-500 mb-4">{error}</p>
          <button
            onClick={fetchPosts}
            className="text-white bg-blue-600 hover:bg-blue-700 font-
medium rounded-lg text-sm px-5 py-2.5"
          >
            Coba Lagi
          </button>
        </div>
      ) : posts.length === 0 ? (
        <div className="text-center py-20">
          <p className="text-gray-500">Belum ada posts.</p>
        </div>
      ) : (
        <div className="grid gap-8 lg:grid-cols-2">
          {posts.map((post) => (
            <PostCard key={post.id} post={post} />
          ))}
        </div>
      )}
    </div>
  </main>

  <Footer />
</div>
);
};

export default PostList;

```

File: frontend-reactjs/src/pages/PostDetail.jsx

```

import { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import { postsAPI } from '../services/api';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import DefaultAvatar from '../components/DefaultAvatar';

const PostDetail = () => {
  const { id } = useParams();
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    fetchPost();
  }, [id]);

  const fetchPost = async () => {
    try {
      setLoading(true);
      const response = await postsAPI.getById(id);
      setPost(response.data.data);
      setError('');
    } catch (err) {
      if (err.response?.status === 404) {
        setError('Post tidak ditemukan.');
```

```

        month: 'long',
        year: 'numeric'
    });
};

if (loading) {
    return (
        <div className="min-h-screen bg-gray-100 flex items-center justify-center">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-
600"></div>
        </div>
    );
}

if (error) {
    return (
        <div className="min-h-screen bg-gray-100 flex flex-col">
            <Navbar />
            <div className="flex-1 flex items-center justify-center py-8 px-4 mx-auto max-
w-screen-xl lg:py-16 lg:px-6">
                <div className="text-center py-20">
                    <p className="text-red-500 mb-4">{error}</p>
                    <Link
                        to="/posts"
                        className="text-white bg-blue-600 hover:bg-blue-700 font-medium
rounded-lg text-sm px-5 py-2.5"
                    >
                        ← Kembali ke Daftar Posts
                    </Link>
                </div>
            </div>
            <Footer />
        </div>
    );
}

return (
    <div className="min-h-screen bg-gray-100 flex flex-col">
        <Navbar />

        <main className="flex-1">
            <div className="py-8 px-4 mx-auto max-w-screen-lg lg:py-16 lg:px-6">
                <div className="mb-6">
                    <Link
                        to="/posts"
                        className="inline-flex items-center text-sm font-medium text-blue-
600 hover:text-blue-500 transition-colors"
                    >
                        <svg className="mr-2 w-4 h-4" fill="currentColor" viewBox="0 0 20
20" xmlns="http://www.w3.org/2000/svg">
                            <path fillRule="evenodd" d="M9.707 16.707a1 1 0 01-1.414 0l-6-
6a1 1 0 010-1.414l6-6a1 1 0 011.414L5.414 9H17a1 1 0 10 2H5.414L4.293 4.293a1 1 0 010
1.414z" clipRule="evenodd"></path>
                        </svg>
                        Back to Posts
                    </Link>
                </div>

                <article className="bg-white rounded-lg border border-gray-200 shadow-md
overflow-hidden">
                    {post.image_url && (
                        <img
                            src={post.image_url}
                            alt={post.title}
                            className="w-full h-64 lg:h-96 object-cover"
                            onError={(e) => {
                                e.target.style.display = 'none';
                            }}
                        />
                    )}

                    <div className="p-6 lg:p-10">
                        <div className="flex flex-wrap items-center gap-4 mb-6">
                            <span className="bg-blue-100 text-blue-800 text-xs font-medium
inline-flex items-center px-2.5 py-0.5 rounded">
                                <svg className="mr-1 w-3 h-3" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">

```

```

<path fillRule="evenodd" d="M2 5a2 2 0 012-2h8a2 2 0
012 2v10a2 2 0 002 2H4a2 2 0 01-2-2V5zm3 1h6v4H5V6zm6 6H5v2h6v-2z" clipRule="evenodd"></path>
0V7z"></path>
</svg>
Article
</span>
<div className="flex items-center space-x-2">
  <DefaultAvatar />
  <span className="font-medium text-sm text-gray-
700">{post.author}</span>
</div>
{post.created_at && (
  <span className="text-sm text-gray-500 flex items-center">
    <svg className="w-4 h-4 mr-1" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
      <path fillRule="evenodd" d="M6 2a1 1 0 00-1
1v1H4a2 2 0 00-2 2v10a2 2 0 002 2h12a2 2 0 002-2V6a2 2 0 00-2-2h-1V3a1 1 0 10-2 0v1H7V3a1 1 0
00-1-1zm0 5a1 1 0 000 2h8a1 1 0 100-2H6z" clipRule="evenodd"></path>
      </svg>
      {formatDate(post.created_at)}
    </span>
  )}
</div>

<h1 className="mb-6 text-3xl lg:text-4xl font-bold tracking-tight
text-gray-900">
  {post.title}
</h1>

<div className="text-gray-600 leading-relaxed text-lg">
  {post.article.split('\n').map((paragraph, index) => (
    <p key={index} className="mb-4">{paragraph}</p>
  ))}
</div>
</div>
</article>
</div>
</main>

<Footer />
</div>
);
};

export default PostDetail;

```

File: backend-ci3/application/config/routes.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

/*
|-----|
| URI ROUTING
|-----|
| This file lets you re-map URI requests to specific controller functions.
|
| Typically there is a one-to-one relationship between a URL string
| and its corresponding controller class/method. The segments in a
| URL normally follow this pattern:
|
|     example.com/class/method/id/
|
| In some instances, however, you may want to remap this relationship
| so that a different class/function is called than the one
| corresponding to the URL.
|
| Please see the user guide for complete details:
|
|     https://codeigniter.com/userguide3/general/routing.html
|
|-----|
| RESERVED ROUTES
|-----|
|

```

```

| There are three reserved routes:
|
|     $route['default_controller'] = 'welcome';
|
| This route indicates which controller class should be loaded if the
| URI contains no data. In the above example, the "welcome" class
| would be loaded.
|
|     $route['404_override'] = 'errors/page_missing';
|
| This route will tell the Router which controller/method to use if those
| provided in the URL cannot be matched to a valid route.
|
|     $route['translate_uri_dashes'] = FALSE;
|
| This is not exactly a route, but allows you to automatically route
| controller and method names that contain dashes. '-' isn't a valid
| class or method name character, so it requires translation.
| When you set this option to TRUE, it will replace ALL dashes in the
| controller and method URI segments.
|
| Examples:  my-controller/index    -> my_controller/index
|            my-controller/my-method -> my_controller/my_method
|
| */
$route['default_controller'] = 'posts';
$route['404_override'] = '';
$route['translate_uri_dashes'] = FALSE;

// Route autentikasi API.
$route['api/auth/register']['post'] = 'api/auth/register';
$route['api/auth/login']['post'] = 'api/auth/login';
$route['api/auth/logout']['post'] = 'api/auth/logout';
$route['api/auth/me']['get'] = 'api/auth/me';

// Route post API diarahkan ke method handle agar bisa diproses berdasarkan HTTP verb.
$route['api/post']['get'] = 'api/post/handle';
$route['api/post']['post'] = 'api/post/handle';
$route['api/post/(:num)']['get'] = 'api/post/handle/$1';
$route['api/post/(:num)']['put'] = 'api/post/handle/$1';
$route['api/post/(:num)']['delete'] = 'api/post/handle/$1';

```

File: backend-ci3/application/modules/Api/controllers/Auth.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Auth extends MX_Controller {

    public function __construct()
    {
        parent::__construct();
        // Memuat model user dan library yang dibutuhkan untuk autentikasi API.
        $this->load->model('User_model');
        $this->load->library('jwt');
        $this->load->library('form_validation');
    }

    /**
     * Endpoint register user baru.
     * POST /api/auth/register
     */
    public function register()
    {
        // Semua response auth dikembalikan dalam format JSON.
        header('Content-Type: application/json; charset=utf-8');
        ob_clean();

        try {
            // Membaca body JSON dari Postman/client.
            $input = json_decode(file_get_contents('php://input'), true);

            // Validasi field wajib untuk proses pendaftaran.
            if (empty($input['name']) || empty($input['email']) || empty($input['password']))
            {
                http_response_code(400);
                echo json_encode([

```

```

        'status' => false,
        'message' => 'Name, email, and password are required'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

// Memastikan format email valid sebelum disimpan.
if (!filter_var($input['email'], FILTER_VALIDATE_EMAIL)) {
    http_response_code(400);
    echo json_encode([
        'status' => false,
        'message' => 'Invalid email format'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

// Email harus unik agar satu akun tidak terdaftar dua kali.
if ($this->User_model->email_exists($input['email'])) {
    http_response_code(409);
    echo json_encode([
        'status' => false,
        'message' => 'Email already registered'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

// Password minimal 6 karakter sesuai kebutuhan praktikum.
if (strlen($input['password']) < 6) {
    // ... bagian kode lain mengikuti pola yang sama ...
    // Token diambil dari header Authorization.
    $token = $this->jwt->get_token_from_request();

    if (!$token) {
        http_response_code(401);
        echo json_encode([
            'status' => false,
            'message' => 'No token provided'
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }

    // Jika token valid, payload token akan berisi ID user.
    $decoded = $this->jwt->verify($token);

    if (!$decoded) {
        http_response_code(401);
        echo json_encode([
            'status' => false,
            'message' => 'Invalid or expired token'
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }

    // Data user asli tetap diambil dari database agar informasinya terbaru.
    $user = $this->User_model->get_by_id($decoded->id);

    if (!$user) {
        http_response_code(404);
        echo json_encode([
            'status' => false,
            'message' => 'User not found'
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }

    http_response_code(200);
    echo json_encode([
        'status' => true,
        'message' => 'User retrieved successfully',
        'data' => [
            'id' => $user->id,
            'name' => $user->name,
            'email' => $user->email,
            'created_at' => $user->created_at,
            'updated_at' => $user->updated_at
        ]
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
}

```

```

        exit;
    } catch (Exception $e) {
        http_response_code(500);
        echo json_encode([
            'status' => false,
            'message' => 'Server error: ' . $e->getMessage()
        ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
        exit;
    }
}
}
}

```

File: backend-ci3/application/modules/Api/controllers/Post.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Post extends MX_Controller {

    public function __construct()
    {
        parent::__construct();
        // Memuat model, JWT, upload, dan helper URL untuk kebutuhan endpoint post.
        $this->load->model('Post_model');
        $this->load->library('jwt');
        $this->load->library('upload');
        $this->load->helper('url');
    }

    /**
     * Membaca multipart/form-data dari php://input.
     * Dipakai karena PHP tidak otomatis mengisi $_POST/$_FILES untuk request PUT.
     */
    private function parse_multipart_form()
    {
        $content_type = $_SERVER['CONTENT_TYPE'] (c) '';
        $input = [];

        // Jika request berbentuk multipart/form-data, pecah body berdasarkan boundary.
        if (strpos($content_type, 'multipart/form-data') !== false) {
            $boundary = '';
            if (preg_match('/boundary=([^\;]+)/', $content_type, $matches)) {
                $boundary = trim($matches[1], '');
            }

            if (!$boundary) {
                return [];
            }

            $raw = file_get_contents('php://input');
            $parts = preg_split('/' . preg_quote("--$boundary") . '/', $raw);

            foreach ($parts as $part) {
                if (empty(trim($part)) || $part === "--\r\n" || $part === "--") {
                    continue;
                }

                // Pisahkan header tiap field dari isinya.
                $split = preg_split("/\r\n\r\n/", trim($part), 2);
                if (count($split) !== 2) {
                    continue;
                }

                $headers = $split[0];
                $content = rtrim($split[1], "\r\n");

                // Ambil nama field, misalnya title, article, atau image.
                if (preg_match('/name="([^\"]+)/', $headers, $matches)) {
                    $name = $matches[1];

                    // Jika terdapat filename, field ini dianggap sebagai upload file.
                    if (preg_match('/filename="([^\"]+)/', $headers, $filename_matches)) {
                        // Ambil Content-Type file jika tersedia.
                        $file_content_type = 'application/octet-stream';
                        if (preg_match('/Content-Type:\s*([^\r\n]+)/', $headers,

```

```

$type_matches)) {
    $file_content_type = trim($type_matches[1]);
}

$input[$name] = [
    'name' => $filename_matches[1],
    'type' => $file_content_type,
    'content' => $content
];
} else {
    // Field biasa selain file.
    $input[$name] = $content;
}
}

return $input;
}

// Fallback untuk body x-www-form-urlencoded.
// ... bagian kode lain mengikuti pola yang sama ...

if (!$post) {
    http_response_code(404);
    echo json_encode([
        'status' => false,
        'message' => 'Post not found'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

// Hapus file gambar jika post memiliki gambar.
if (!empty($post->image)) {
    $file = $this->get_image_path($post->image);
    if (file_exists($file)) {
        unlink($file);
    }
}

// Hapus data post dari database.
$result = $this->Post_model->delete($id);

if (!$result) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Failed to delete post'
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

http_response_code(200);
echo json_encode([
    'status' => true,
    'message' => 'Post deleted successfully'
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
exit;

} catch (Exception $e) {
    http_response_code(500);
    echo json_encode([
        'status' => false,
        'message' => 'Server error: ' . $e->getMessage()
    ], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
    exit;
}

}

/**
 * Menyimpan konten file sementara dari multipart PUT.
 */
private function _save_temp_file($content)
{
    $temp_path = sys_get_temp_dir() . '/ci3_upload_' . uniqid() . '.tmp';
    file_put_contents($temp_path, $content);
    return $temp_path;
}

```

```

private function _upload_image()
{
    // Konfigurasi upload gambar untuk post.
    $config['upload_path'] = FCPATH . 'uploads/posts/';
    $config['allowed_types'] = 'gif|jpg|png|jpeg';
    $config['max_size'] = 2048; // 2MB
    $config['file_name'] = 'post_' . time() . '_' . uniqid();

    // Buat folder upload jika belum tersedia.
    if (!is_dir($config['upload_path'])) {
        mkdir($config['upload_path'], 0755, true);
    }

    $this->upload->initialize($config);

    if (!$this->upload->do_upload('image')) {
        return false;
    }

    $upload_data = $this->upload->data();
    return $upload_data['file_name'];
}
}

```

File: backend-ci3/application/modules/Api/models/User_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class User_model extends CI_Model {

    // Nama tabel untuk menyimpan data register dan login.
    private $table = 'users';

    public function __construct()
    {
        parent::__construct();
        $this->load->database();
    }

    /**
     * Mengambil seluruh data user.
     */
    public function get_all()
    {
        $query = $this->db->get($this->table);
        return $query->result();
    }

    /**
     * Mengambil user berdasarkan ID dari token atau parameter.
     */
    public function get_by_id($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->row();
    }

    /**
     * Mengambil user berdasarkan email untuk proses login.
     */
    public function get_by_email($email)
    {
        $query = $this->db->get_where($this->table, array('email' => $email));
        return $query->row();
    }

    /**
     * Menyimpan user baru hasil register.
     */
    public function create($data)
    {
        $data['created_at'] = date('Y-m-d H:i:s');
        $data['updated_at'] = date('Y-m-d H:i:s');
    }
}

```



```

        $this->db->insert($this->table, $data);
        return $this->db->insert_id();
    }

    /**
     * Mengubah data user.
     */
    public function update($id, $data)
    {
        $data['updated_at'] = date('Y-m-d H:i:s');
        $this->db->where('id', $id);
        return $this->db->update($this->table, $data);
    }

    /**
     * Menghapus data user.
     */
    public function delete($id)
    {
        $this->db->where('id', $id);
        return $this->db->delete($this->table);
    }

    /**
     * Mengecek apakah email sudah pernah terdaftar.
     */
    public function email_exists($email)
    {
        $query = $this->db->get_where($this->table, array('email' => $email));
        return $query->num_rows() > 0;
    }

    /**
     * Mengecek apakah user tersedia berdasarkan ID.
     */
    public function exists($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->num_rows() > 0;
    }
}

```

File: backend-ci3/application/modules/Api/models/Post_model.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Post_model extends CI_Model {

    // Nama tabel untuk menyimpan artikel/post.
    private $table = 'posts';

    public function __construct()
    {
        parent::__construct();
        $this->load->database();
    }

    /**
     * Mengambil semua post dengan pagination.
     */
    public function get_all($limit = 10, $offset = 0)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->limit($limit, $offset)
            ->order_by('created_at', 'DESC')
            ->get($this->table);
        return $query->result();
    }

    /**
     * Menghitung total post untuk informasi pagination.
     */
    public function count_all()
    {

```

```

        return $this->db->count_all($this->table);
    }

    /**
     * Mengambil satu post berdasarkan ID.
     */
    public function get_by_id($id)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->get_where($this->table, array('id' => $id));
        return $query->row();
    }

    /**
     * Menyimpan post baru.
     */
    public function create($data)
    {
        $data['created_at'] = date('Y-m-d H:i:s');
        $data['updated_at'] = date('Y-m-d H:i:s');

        $this->db->insert($this->table, $data);
        return $this->db->insert_id();
    }

    /**
     * Mengubah data post.
     */
    public function update($id, $data)
    {
        $data['updated_at'] = date('Y-m-d H:i:s');
        $this->db->where('id', $id);
        return $this->db->update($this->table, $data);
    }

    /**
     * Menghapus data post.
     */
    public function delete($id)
    {
        $this->db->where('id', $id);
        return $this->db->delete($this->table);
    }

    /**
     * Mengecek apakah post tersedia berdasarkan ID.
     */
    public function exists($id)
    {
        $query = $this->db->get_where($this->table, array('id' => $id));
        return $query->num_rows() > 0;
    }

    /**
     * Mencari post berdasarkan kata kunci.
     */
    public function search($keyword, $limit = 10, $offset = 0)
    {
        $this->db->like('title', $keyword);
        $this->db->or_like('description', $keyword);
        $this->db->limit($limit, $offset);
        $this->db->order_by('created_at', 'DESC');
        $query = $this->db->get($this->table);
        return $query->result();
    }

    /**
     * Mengambil post berdasarkan user jika relasi user dipakai.
     */
    public function get_by_user($user_id, $limit = 10, $offset = 0)
    {
        $query = $this->db
            ->select('id, title, author, article, image, created_at, updated_at')
            ->where('user_id', $user_id)
            ->limit($limit, $offset)
            ->order_by('created_at', 'DESC')

```

```

        ->get($this->table);
        return $query->result();
    }
}

```

File: backend-ci3/application/libraries/Jwt.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

class Jwt
{
    // Secret key dipakai untuk membuat dan memverifikasi tanda tangan token.
    private $secret;
    private $algo = 'sha256';
    // Token berlaku 24 jam.
    private $ttl = 86400;

    public function __construct()
    {
        // Ambil secret dari config agar mudah diganti tanpa mengubah library.
        $CI =& get_instance();
        $key = $CI->config->item('jwt_secret_key');

        if (empty($key)) {
            $key = $CI->config->item('encryption_key');
        }

        $this->secret = !empty($key) ? $key : 'ci3-hmvc-rest-api-secret';
    }

    public function create($payload = array())
    {
        // Payload ditambah waktu pembuatan dan waktu kedaluwarsa token.
        $now = time();
        $payload['iat'] = isset($payload['iat']) ? $payload['iat'] : $now;
        $payload['exp'] = isset($payload['exp']) ? $payload['exp'] : ($now + $this->ttl);

        // JWT terdiri dari header, payload, dan signature.
        $header = array('typ' => 'JWT', 'alg' => 'HS256');
        $segments = array(
            $this->base64url_encode(json_encode($header)),
            $this->base64url_encode(json_encode($payload))
        );

        $signature = hash_hmac($this->algo, implode('.', $segments), $this->secret, true);
        $segments[] = $this->base64url_encode($signature);

        return implode('.', $segments);
    }

    public function verify($token)
    {
        // Token valid harus memiliki tiga bagian yang dipisahkan titik.
        $parts = explode('.', $token);

        if (count($parts) !== 3) {
            throw new Exception('Invalid token format');
        }

        // Signature dihitung ulang untuk memastikan token tidak dimodifikasi.
        list($header64, $payload64, $signature64) = $parts;
        $signature = $this->base64url_decode($signature64);
        $expected = hash_hmac($this->algo, $header64 . '.' . $payload64, $this->secret, true);

        if (!$this->hash_equals($expected, $signature)) {
            throw new Exception('Invalid token signature');
        }

        $payload = json_decode($this->base64url_decode($payload64));

        if (!$payload) {
            throw new Exception('Invalid token payload');
        }

        // Token ditolak jika sudah melewati waktu kedaluwarsa.
    }
}

```

```

        if (isset($payload->exp) && time() > $payload->exp) {
            throw new Exception('Token has expired');
        }

        return $payload;
    }

    public function get_token_from_request()
    {
        // Token dibaca dari header Authorization dengan format Bearer <token>.
        $headers = function_exists('getallheaders') ? getallheaders() : array();
        $authorization = null;

        foreach ($headers as $name => $value) {
            if (strtolower($name) === 'authorization') {
                $authorization = $value;
                break;
            }
        }

        if (empty($authorization) && !empty($_SERVER['HTTP_AUTHORIZATION'])) {
            $authorization = $_SERVER['HTTP_AUTHORIZATION'];
        }

        if (empty($authorization) && !empty($_SERVER['REDIRECT_HTTP_AUTHORIZATION'])) {
            $authorization = $_SERVER['REDIRECT_HTTP_AUTHORIZATION'];
        }

        if (!empty($authorization) && preg_match('/Bearer\s+(.*)$/i', $authorization,
$matches)) {
            return trim($matches[1]);
        }

        return null;
    }

    private function base64url_encode($data)
    {
        // Format base64url dipakai oleh standar JWT.
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    private function base64url_decode($data)
    {
        $remainder = strlen($data) % 4;

        if ($remainder) {
            $data .= str_repeat('=', 4 - $remainder);
        }

        return base64_decode(strtr($data, '-_', '+/'));
    }

    private function hash_equals($known, $user)
    {
        // Fallback untuk membandingkan signature pada versi PHP lama.
        if (function_exists('hash_equals')) {
            return hash_equals($known, $user);
        }

        if (strlen($known) !== strlen($user)) {
            return false;
        }

        $result = 0;
        for ($i = 0; $i < strlen($known); $i++) {
            $result |= ord($known[$i]) ^ ord($user[$i]);
        }

        return $result === 0;
    }
}

```

File: backend-ci3/ci3_project.sql

```

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET NAMES utf8 */;
/*!50503 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

DROP DATABASE IF EXISTS `ci3_project`;
CREATE DATABASE IF NOT EXISTS `ci3_project` /*!40100 DEFAULT CHARACTER SET utf8mb4 COLLATE
utf8mb4_0900_ai_ci */ /*!80016 DEFAULT ENCRYPTION='N' */;
USE `ci3_project`;

CREATE TABLE IF NOT EXISTS `posts` (
  `id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `author` varchar(255) NOT NULL,
  `article` text NOT NULL,
  `image` varchar(255) DEFAULT NULL,
  `created_at` timestamp NOT NULL DEFAULT (now()),
  `updated_at` timestamp NOT NULL DEFAULT (now()),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=2 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

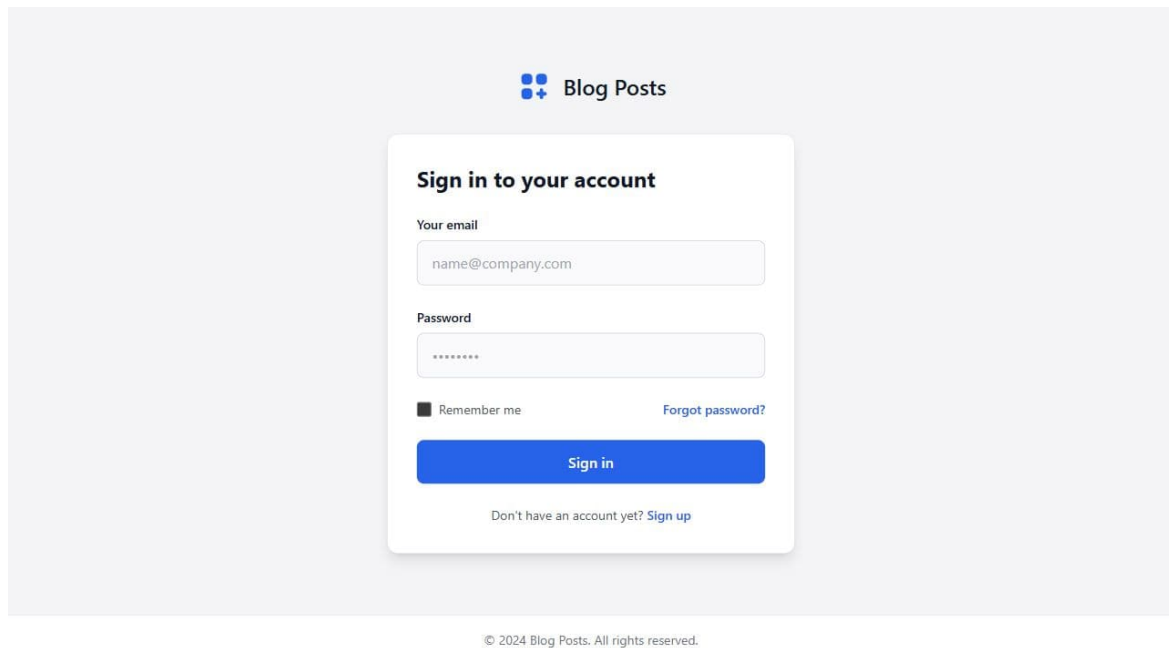
CREATE TABLE IF NOT EXISTS `users` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `created_at` timestamp NOT NULL DEFAULT (now()),
  `updated_at` timestamp NOT NULL DEFAULT (now()),
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_email_unique` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

INSERT INTO `posts` (`id`, `title`, `author`, `article`, `image`, `created_at`, `updated_at`)
VALUES
  (1, 'tes post', 'dhodho', 'lorem ipsum blablabla',
  'posts/1777305501_telkom-university_logo_freelogovectors_net_.png', '2026-04-27 08:58:21',
  '2026-04-27 08:58:21');

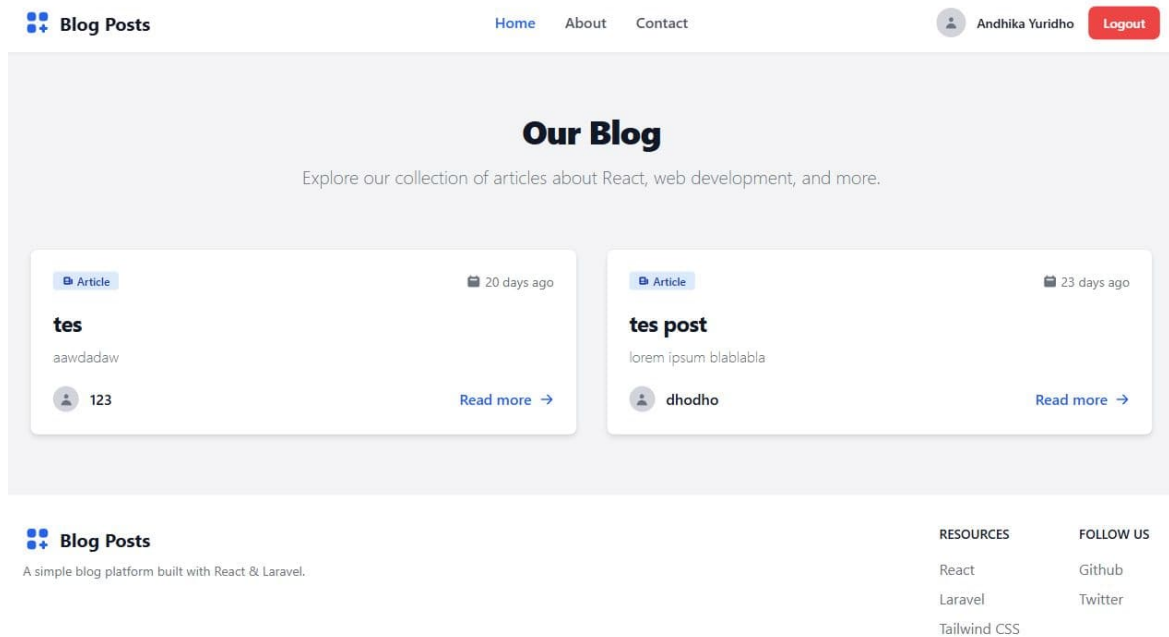
/*!40103 SET TIME_ZONE=IFNULL(@OLD_TIME_ZONE, 'system') */;
/*!40101 SET SQL_MODE=IFNULL(@OLD_SQL_MODE, '') */;
/*!40014 SET FOREIGN_KEY_CHECKS=IFNULL(@OLD_FOREIGN_KEY_CHECKS, 1) */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40111 SET SQL_NOTES=IFNULL(@OLD_SQL_NOTES, 1) */;

```

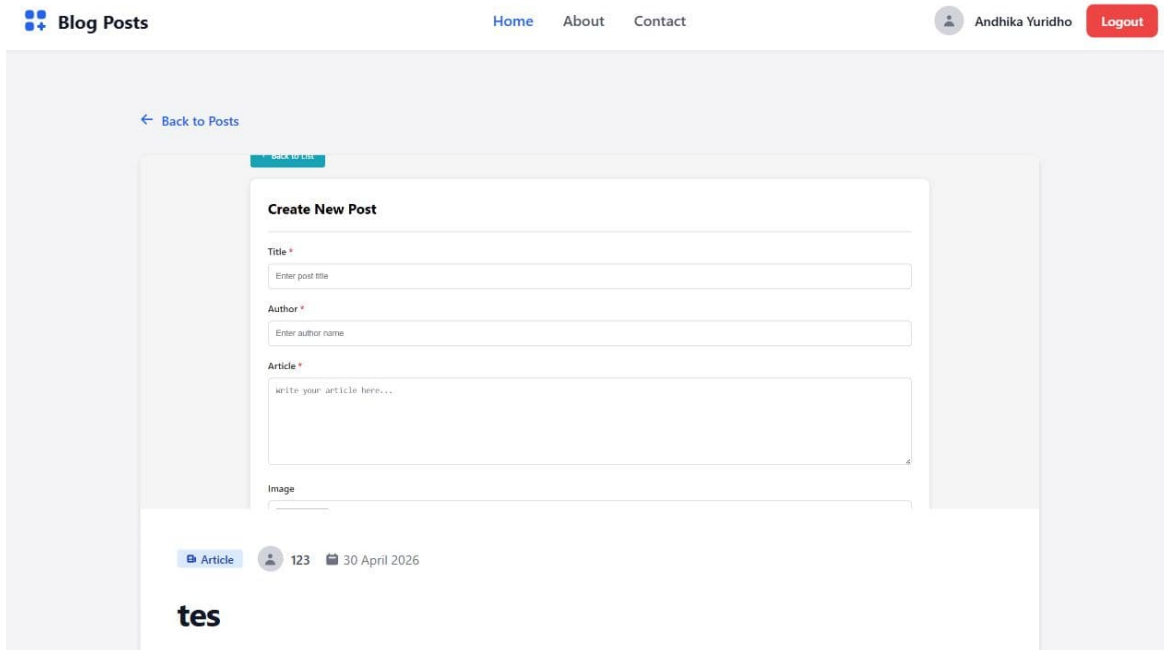
Output



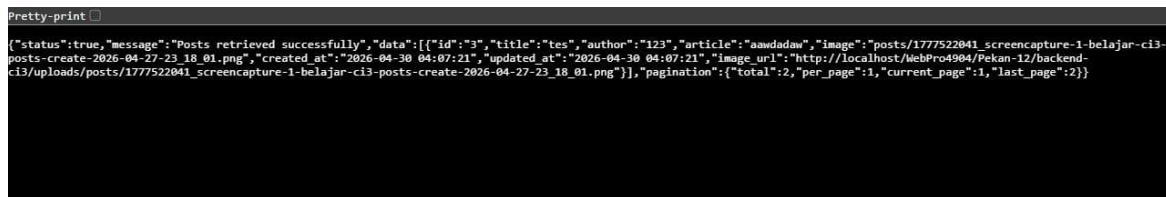
Gambar 1. Tampilan halaman login aplikasi React Blog Post



Gambar 2. Tampilan daftar blog post setelah user berhasil login



Gambar 3. Tampilan detail blog post yang diambil dari API



```

{"status":true,"message":"Posts retrieved successfully","data":[{"id":"3","title":"tes","author":"123","article":"aawdadaw","image":"posts/177522041_screenshot-1-belajar-ci3-posts-create-2026-04-27-23_18_01.png","created_at":"2026-04-30 04:07:21","updated_at":"2026-04-30 04:07:21","image_url":"http://localhost/WebPro4904/Pekan-12/backend-ci3/uploads/posts/177522041_screenshot-1-belajar-ci3-posts-create-2026-04-27-23_18_01.png"}],"pagination":{"total":2,"per_page":1,"current_page":1,"last_page":2}}

```

Gambar 4. Respons JSON endpoint API post pada browser

Analisis dan Pembahasan

Praktikum ini menghubungkan aplikasi React dengan backend REST API CodeIgniter 3. React berperan sebagai frontend yang menampilkan antarmuka pengguna, sedangkan CodeIgniter 3 menyediakan data dalam format JSON melalui endpoint /api. Pemisahan ini membuat frontend dan backend dapat dikembangkan secara lebih terstruktur.

Pada sisi frontend, file services/api.js menggunakan Axios sebagai pusat komunikasi API. Interceptor request menambahkan token Bearer dari localStorage jika tersedia, sedangkan interceptor response menangani kondisi unauthorized dengan menghapus token dan mengarahkan user ke halaman login. Dengan cara ini, logika komunikasi API tidak perlu ditulis ulang pada setiap halaman.

AuthContext digunakan untuk menyimpan status autentikasi user. Setelah login berhasil, token dan data user disimpan pada localStorage, kemudian route protected dapat membaca status tersebut untuk menentukan apakah user boleh membuka halaman posts. Halaman PostList dan PostDetail mengambil data dari endpoint post, lalu menampilkannya dalam bentuk card dan halaman artikel.

Pada sisi backend, CodeIgniter 3 menggunakan module Api yang berisi controller Auth dan Post. Controller Auth menangani register, login, logout, dan data user aktif, sedangkan controller Post menangani operasi data artikel. Library Jwt dipakai untuk membuat token login dan memverifikasi request yang membutuhkan autentikasi.

Hasil pengujian menunjukkan bahwa endpoint API dapat mengembalikan data JSON dan frontend dapat menampilkan data post dari backend. Dengan demikian, integrasi React dan CodeIgniter 3 telah berjalan sesuai tujuan praktikum.

Kesimpulan

Praktikum berhasil diimplementasikan. Aplikasi React Blog Post dapat terhubung dengan REST API CodeIgniter 3, melakukan login menggunakan token JWT, mengambil daftar post, menampilkan detail post, serta membaca respons JSON dari endpoint API. Integrasi ini menunjukkan bahwa React dapat digunakan sebagai frontend modern untuk mengonsumsi data dari backend berbasis CodeIgniter 3.

Lampiran

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/AndhikaYuridho-707012500059>